

24. Reading and writing text files

Herman Kamper

Up to now, every piece of data in our programs has lived in a variable or an array, and every one of those disappears the moment the program ends. Real programs are not like that. They load records when they start and save results when they finish: almost every program you use reads and writes files, and any collection of records you care about is stored in one.

In this lecture we look at the simplest kind: text files. We will see how to get the lines of a text file into variables in C, where everything you have learnt so far can be used on them, and how to go the other way and create a new text file with the results.

Why files?

Suppose you want to build a program that recommends movies. Public datasets like the TMDb 5000 Movie Dataset have exactly the raw material for this: for about five thousand movies they record who starred in the movie, who directed it, how long it is, what genre it belongs to and how much money it made.¹

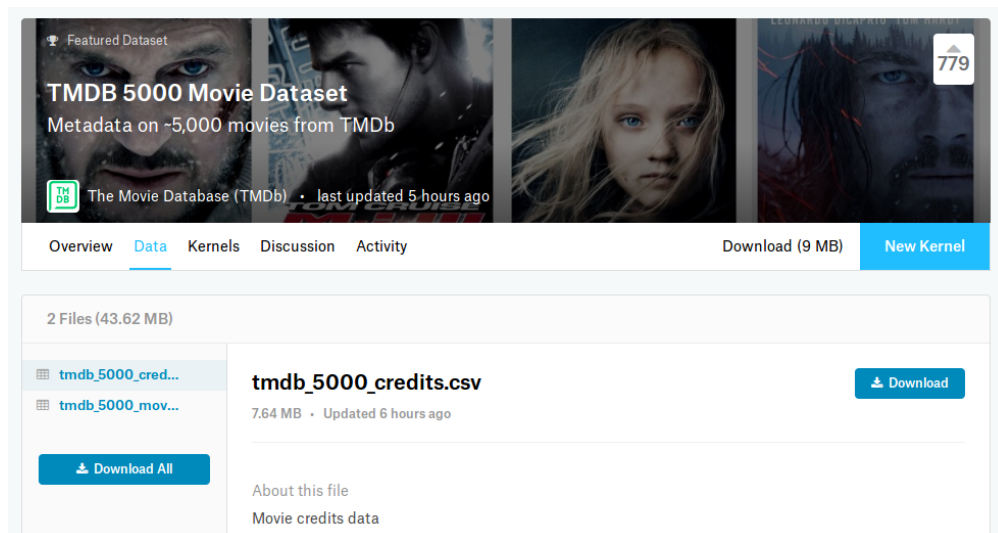


Figure 1: The TMDb 5000 Movie Dataset on Kaggle. The data comes as a single file, `tmdb_5000_credits.csv`, which is several megabytes of plain text.

With data like this you can build a model that tells you how similar two movies are. Streaming

¹The dataset is hosted on [Kaggle](#), a website that hosts machine learning competitions.

services use something like it: if you have watched five movies, they find the sixth most similar one and put it at the top of your page, and that is how a quiet evening turns into seven hours of watching. It is a slightly evil use of the data, but it is a good example for learning something useful.

Whatever you want to do with the data, the first problem is the same. The data is in a file, your program works with variables, and you need a way to load the one into the other. That is what we do here.

The format of the file in the figure is not important for us. The file ends in `.csv`, which stands for comma-separated values: a text file in which each line is one record and the fields in the record are separated by commas. A smaller file in the same spirit will do for our examples. Create a text file called `movies.txt` containing these three lines:

```
#budget,genres,homepage,title
237000000,action,www.avatarmovie.com/,Avatar
300000000,fantasy,http://disney.go.com/disneypictures/pirates/,Pirates
```

The first line is a header, saying what each field is. The `#` at the start is just a convention that marks it as a header rather than a record. Put the file in the same directory as your program: when you give `fopen` a plain file name, that is where it looks. If the file lives somewhere else, you give the whole path in the file name instead.

Opening and closing a file

To work with a file we need a way to refer to it. In C that way is a pointer to a structure called `FILE`:

```
FILE *fPtr;
```

You do not need to know what is inside a `FILE` structure for now. Think of it as a handle: an object in memory that keeps track of one open file for you, including how far through the file you have got. The pointer `fPtr` is your hold on that handle, and every function that reads from or writes to the file will want it as an argument.

The function that creates the handle and connects it to a file on disk is `fopen`, which lives in `stdio.h`. It takes two arguments: the name of the file, and a single-character mode that says what you intend to do with it. Here we only want to read, so the mode is `"r"`:

```
fPtr = fopen("movies.txt", "r");
```

When you are finished with the file, you disconnect from it with `fclose`:

```
fclose(fPtr);
```

Get into the habit of writing the `fclose` as soon as you write the `fopen`. For reading it mostly tidies up, but when you are writing it matters a lot: output is often held in memory for a while before it reaches the disk, and closing the file is what makes sure it gets there.

That is the skeleton of every file program: open, do something, close. By the end of this lecture we will have these functions in our toolbox:

Function	What it does
<code>fopen</code>	opens a file and gives you a handle for it
<code>fclose</code>	closes the file
<code>fgets</code>	reads one line from a file into a string
<code>fscanf</code>	reads formatted values from a file, like <code>scanf</code>
<code>fprintf</code>	writes formatted values to a file, like <code>printf</code>
<code>feof</code>	checks whether you have run off the end of the file
<code>rewind</code>	goes back to the start of the file

Reading a line with `fgets`

We now want the lines of `movies.txt` in our program. A line is a string, so first we need somewhere to store it: a character array. Make it 150 characters, so a long line will still fit:

```
char line[150];
```

You already know how to read a string from the user: `scanf`. Reading from a file works the same way, except you have to say which file to read from, so there is a version with an `f` in front of it. But for reading a whole line there is a better function, `fgets` (get string):

```
fgets(line, 150, fPtr);
```

It takes the array to read into, the size of that array (so it can never write past the end of it)² and the file handle. Here is a complete program that reads the first line of the file and prints it:

```
#include <stdio.h>

int main()
{
    FILE *fPtr;
    char line[150];

    fPtr = fopen("movies.txt", "r");

    fgets(line, 150, fPtr);
    printf("The first line is: %s", line);

    fclose(fPtr);

    return 0;
}
```

Running it prints:

```
The first line is: #budget,genres,homepage,title
```

²Strictly, `fgets` stores at most one fewer character than this number, because it always adds the string-terminating `\0`.

Note that `fgets` keeps the newline character at the end of the line, which is why the `printf` has no `\n` of its own. If you add one, you get a blank line after every line you print.

What does the handle do after you have read a line? It moves on to the start of the next line, automatically. A second call to `fgets` therefore reads the second line, a third reads the third, and each one overwrites whatever was in line before. Think of it like this, with an arrow showing where the handle is pointing:

```
after fopen:          -> #budget,genres,homepage,title
                        2370000000,action,www.avatarmovie.com/,Avatar
                        3000000000,fantasy,http://disney.go.com/...

after 1st fgets:      #budget,genres,homepage,title
                        -> 2370000000,action,www.avatarmovie.com/,Avatar
                        3000000000,fantasy,http://disney.go.com/...

after 2nd fgets:      #budget,genres,homepage,title
                        2370000000,action,www.avatarmovie.com/,Avatar
                        -> 3000000000,fantasy,http://disney.go.com/...
```

After the first `fgets`, the array line holds `#budget,genres,homepage,title\n` followed by the string-terminating `\0`. After the second, all of that has been wiped out and line holds the Avatar line. The array lives in memory and is entirely separate from the file on disk. Opening a file in "r" mode always starts at the beginning of the file, and with the functions we have so far, you can only move forwards, one line at a time.

Reading formatted values with `fscanf`

`fgets` gives you a line as one string. Often you want something more: a name here and a number there, with each converted into the right type. For this there is `fscanf`, which is exactly `scanf` with the file handle added as a first argument.

Suppose we have a file `birthyears.txt` with a name and a year on each line:

```
Herman 1987
Phia 2024
```

We declare a string for the name and an integer for the year, then read both from the first line in one go:

```

#include <stdio.h>

int main()
{
    FILE *fPtr;
    char name[50];
    int year;

    fPtr = fopen("birthyears.txt", "r");

    fscanf(fPtr, "%s %d", name, &year);
    printf("%s is born in year %d\n", name, year);

    fscanf(fPtr, "%s %d", name, &year);
    printf("%s is born in year %d\n", name, year);

    fclose(fPtr);

    return 0;
}

```

The program prints:

```

Herman is born in year 1987
Phia is born in year 2024

```

Everything here is what you already know from `scanf`. The `%s` reads a string into `name`, which needs no `&` because an array name already is an address. The `%d` reads an integer, which does need the `&` so that `fscanf` can reach the box. The only new thing is the file handle at the front. As with `fgets`, the handle moves on after each call, so the second `fscanf` picks up where the first left off and overwrites `name` and `year` with the values from the second line.

There is one important difference between `fscanf` and `fgets` when it comes to strings. A `%s` stops at the first whitespace character, so it reads one word, not one line. If we used it on the third line of `movies.txt`, it would stop after `Pirates`, at the first space in the title. Reading the same file with three calls of `fscanf(fPtr, "%s", line)` would give these three strings, and the rest of each line would still be waiting for the next call:

```

#budget,genres,homepage,title
237000000,action,www.avatarmovie.com/,Avatar
300000000,fantasy,http://disney.go.com/disneypictures/pirates/,Pirates

```

The first two lines have no spaces in them, so they come out whole. The third one is cut off. When you want to read a whole line as a single string, `fgets` is the right tool, and it reads right up to the end of the line. When you want to pick values out of a line and convert them to numbers, `fscanf` is the right tool. That is also why there are two of them: `fscanf` is a general-purpose reader that can fill any type, whereas `fgets` only reads strings but does that job better.

Reading until the end of the file

Often you do not know in advance how many lines a file has. For three lines we could simply call `fgets` three times, but for the TMDB file we would want to keep reading, line by line, until the file runs out. For this we need a way to ask whether we have reached the end. The function `feof` (file end-of-file) does that: it returns true once the end of the file has been reached.

That suggests a loop. Here is a first attempt: read the first line, then keep going while we are not at the end of the file, reading a line and printing it each time.

```
fgets(line, 150, fPtr);
printf("The first line is: %s", line);

while (!feof(fPtr))
{
    fgets(line, 150, fPtr);
    printf("The next line is: %s", line);
}
```

Run on our three-line file, this prints:

The first line is: #budget,genres,homepage,title

The next line is: 237000000,action,www.avatarmovie.com/,Avatar

The next line is: 300000000,fantasy,http://disney.go.com/disneypictures/pirates/,Pirates

The next line is: 300000000,fantasy,http://disney.go.com/disneypictures/pirates/,Pirates

The last line appears twice. To see why, you have to know what `feof` really checks. It does not tell you that the handle is pointing at the last line, or even just past it. It only becomes true after a read has *tried* to go past the end of the file and failed. Let us step through:

1. The first `fgets` reads the header. The handle moves to the Avatar line.
2. `feof` is false, so we enter the loop. We read the Avatar line and print it. The handle moves to the Pirates line.
3. `feof` is still false. We read the Pirates line and print it. The handle is now just past the last character of the file, but nobody has yet tried to read from there, so `feof` is still false.
4. We enter the loop a third time. This `fgets` tries to read, finds nothing and fails, so it leaves `line` untouched, with the Pirates line still in it. The failed read is what finally sets the end-of-file indicator. The `printf` then prints the stale contents of `line` for the second time.
5. `feof` is now true, and the loop ends.

The fix is to put the read and the print in the other order: print what you have just read, then read the next line:

```
fgets(line, 150, fPtr);

while (!feof(fPtr))
{
    printf("%s", line);
    fgets(line, 150, fPtr);
}
```

Now the final `fgets`, the one that fails, is directly followed by the loop condition and not by a `printf`, so nothing is printed twice. Compare the two versions carefully, because the shape of this mistake is very common.

If all this feels slightly confusing, hold on to one sentence: `feof` becomes true only after you have tried to read past the end of the file.

There is a way of writing the loop that avoids the subtlety altogether. The function `fgets` returns `NULL` when it fails to read anything, so you can test the return value itself and never ask `feof` at all:

```
while (fgets(line, 150, fPtr) != NULL)
{
    printf("%s", line);
}
```

This reads a line and checks that it worked in one step, so there is no stale line to print. You will see both styles in other people's code, so you need to be able to read both.

Going back to the start

Sometimes you want to read a file more than once, for instance to count the lines in one pass and then process them in the next. The handle only moves forward, so we need a way to send it back to the beginning. That is `rewind`:

```
rewind(fPtr);
```

After a `rewind`, the next read starts at the first line again. This program reads the first line twice:

```
fgets(line, 150, fPtr);
printf("%s", line);
```

```
rewind(fPtr);
```

```
fgets(line, 150, fPtr);
printf("%s", line);
```

It prints the header line two times. There are other functions that let you jump to a particular position in a file; we do not need them yet.

Writing to a file

Writing is the mirror image of reading. We open the file, put things in it and close it. The function you use for putting things in is `fprintf`, which is `printf` with a file handle in front, just as `fscanf` is `scanf` with one:

```
fprintf(fPtr, "The Notebook,Noah\n");
```

Everything you can do with `printf` you can do with `fprintf`. You can use `%d`, `%f`, `%s` and the rest of the format specifiers, and you can print as many values as you like. All that changes is where the output goes: into the file instead of onto the screen.

The only new thing is the mode. To write we open the file with `"w"`:

```
fPtr = fopen("new_movie_file.txt", "w");
```

If the file does not exist, `fopen` creates it. Just opening it is already enough to bring an empty file into existence: you can run a program that does nothing but `fopen` and `fclose` in `"w"` mode and find an empty file next to it afterwards.

Here is a complete program that writes two records to a new file:

```
#include <stdio.h>

int main()
{
    FILE *fPtr;

    fPtr = fopen("new_movie_file.txt", "w");

    fprintf(fPtr, "The Notebook,Noah\n");
    fprintf(fPtr, "Spiderman Brand New Day,Peter Parker\n");

    fclose(fPtr);

    return 0;
}
```

After running it, open `new_movie_file.txt` in a text editor and you will see:

```
The Notebook,Noah
Spiderman Brand New Day,Peter Parker
```

That is the whole story for writing: a record per `fprintf`, and a `\n` at the end of each one so that the records end up on separate lines.

There is a trap in the `"w"` mode, and it is worth stating plainly. If a file with that name already exists, opening it with `"w"` throws away everything that was in it and starts again from empty. It is a very efficient way to lose all of your records. If instead you want to keep what is there and add to the end, use `"a"`, which stands for append:

```
fPtr = fopen("new_movie_file.txt", "a");
```

Running the program above twice with "a" leaves four lines in the file, whereas with "w" it would leave two.

The open modes

We have now seen three modes, "r", "w" and "a". These are the ones you will use nearly all of the time. For completeness, here are all the text-file modes:

Mode	Meaning
"r"	Open a file for reading.
"w"	Create a file for writing. If the file already exists, discard the current contents.
"a"	Append: open or create a file for writing at the end of the file.
"r+"	Open a file for update (reading and writing).
"w+"	Create a file for update. If the file already exists, discard the current contents.
"a+"	Append: open or create a file for update, with writing done at the end of the file.

Each of these has a binary equivalent, with a b added: "rb", "wb", "ab", "rb+", "wb+" and "ab+". We meet these in the next lecture, along with the difference between text files and binary files. The + modes are a more specialised tool that lets you both read and write through one handle. You can get a long way without them, but it is worth reading the table until you can say what each of the six does, because you may well meet them in somebody else's code.

When the file does not open

Everything so far has quietly assumed that `fopen` succeeds. It does not always. If you try to read from a file that does not exist, the operating system cannot open it. When you write, the disk may be full, or the file name may include a directory that does not exist. In each of these cases `fopen` returns `NULL` instead of a valid handle.

That makes the check easy, and it is good practice to always do it before touching the file:

```
fPtr = fopen("movies.txt", "r");

if (fPtr == NULL)
{
    printf("ERROR - file could not be opened!\n");
}
else
{
    /* ... read from the file ... */

    fclose(fPtr);
}
```

Without this check, a missing file leads your program to call `fgets` or `fscanf` on a handle that does not exist, and it will most likely crash. With it, you can print a message and stop cleanly.

Special files: `stdin`, `stdout` and `stderr`

When a program starts, three files are already open for it, and you did not have to open them. They are called `stdin`, `stdout` and `stderr`, they are of type `FILE *`, and they are declared in `stdio.h`.

Name	What it is
<code>stdin</code>	standard input: the keyboard
<code>stdout</code>	standard output: the screen
<code>stderr</code>	standard error: also the screen, but for error messages

They are not proper files on disk, but they behave like them, and that is why `scanf` and `printf` can be seen as special cases of `fscanf` and `fprintf`. These two lines do exactly the same thing:

```
printf("Please enter filename: ");
fprintf(stdout, "Please enter filename: ");
```

And so do these two:

```
scanf("%29s", fileName);
fscanf(stdin, "%29s", fileName);
```

Nobody writes the longer versions on purpose, and you should not, but you may come across a programmer who does, and now you know what `stdout` and `stdin` mean when you see them.

The third file, `stderr`, is useful in its own right. It is also displayed on the screen, but it is a separate stream, intended for error messages. Programs commonly write their results to `stdout` and their complaints to `stderr`, so that the two can be told apart or sent to different places. The error message from the last section is a natural candidate:

```
fprintf(stderr, "ERROR - file could not be opened!\n");
```

Putting it together

Let us finish with two complete programs that use almost everything from this lecture. The first writes to a file whose name the user chooses:

```
#include <stdio.h>

int main(void)
{
    FILE *fPtr;
    char fileName[30];
    char lineOfText[50] = "A single line of text";
    int myInt = 5;
    float myFloat = 1.345;

    printf("Please enter filename: ");
    scanf("%29s", fileName);

    fPtr = fopen(fileName, "w");

    if (fPtr == NULL)
    {
        printf("ERROR - File could not be opened!\n");
    }
    else
    {
        fprintf(fPtr, "*");
        fprintf(fPtr, "%s", lineOfText);
        fprintf(fPtr, "*");
        fprintf(fPtr, "\nmyInt = %d\nmyFloat = %.3f\n", myInt, myFloat);
        fclose(fPtr);
    }

    return 0;
}
```

The `%29s` leaves room for the string-terminating `\0` in the 30-character array. Before reading

on, work out what this puts in the file. The three `fprintf` calls at the top write a `*`, then the string, then another `*`, all on one line. The last one starts with `\n`, which ends that line, and then writes two more lines. The file contains:

```
*A single line of text*
myInt = 5
myFloat = 1.345
```

The second program reads a file like that. Read it carefully and see whether you can say what it prints for this file before you look at the answer:

```
#include <stdio.h>

int main(void)
{
    FILE *fPtr;
    char fileName[30];
    char lineOfText[30], string1[20], string2[20];
    float myFloat;

    printf("Please enter filename: ");
    scanf("%29s", fileName);

    fPtr = fopen(fileName, "r");

    if (fPtr == NULL)
    {
        printf("ERROR - file could not be opened!\n");
    }
    else
    {
        fgets(lineOfText, 29, fPtr);
        while (!feof(fPtr))
        {
            printf("%s", lineOfText);
            fgets(lineOfText, 29, fPtr);
        }

        rewind(fPtr);

        printf("%c\n", fgetc(fPtr));
        fgets(lineOfText, 29, fPtr);
        printf(" %s", lineOfText);

        fgets(lineOfText, 29, fPtr);
        printf("%s", lineOfText);
    }
}
```

```

        fscanf(fPtr, "%19s%19s%f", string1, string2, &myFloat);
        printf("Value of myFloat = %.3f\n", myFloat);

        fclose(fPtr);
    }

    return 0;
}

```

There is one function in there that we have not seen: `fgetc` reads a single character from the file and moves the handle on by one character. With the file above, the program prints:

```

*A single line of text*
myInt = 5
myFloat = 1.345
*
  A single line of text*
myInt = 5
Value of myFloat = 1.345

```

The first three lines come from the loop that reads the file from start to finish. After the `rewind`, `fgetc` reads just the leading `*`, so the handle is now part-way through the first line, and the `fgets` that follows reads the *rest* of that line, which we print with a space in front of it. The next `fgets` reads the `myInt` line. That leaves the handle at the start of the `myFloat` line, where `fscanf` reads two strings, `myFloat` and `=`, and then the number 1.345 into a float. Each of these steps uses only what we have covered, and they are all just different ways of moving through the same file.

Summary

A file in C is accessed through a handle, a pointer of type `FILE *`. The same few steps cover almost everything you will do with text files:

- Open the file with `fopen(name, mode)`. Use `"r"` to read, `"w"` to write (discarding any existing contents) and `"a"` to append. Check that the result is not `NULL`.
- Read with `fgets` for a whole line as a string, or `fscanf` for formatted values, exactly like `scanf` with a file handle in front.
- Write with `fprintf`, exactly like `printf` with a file handle in front.
- Every read moves the handle forward. `rewind` takes it back to the start of the file.
- `feof` only becomes true after a read has gone past the end of the file, so read first and test afterwards, or loop on the return value of `fgets`.
- Close the file with `fclose` when you are done.
- `stdin`, `stdout` and `stderr` are three files that are always open: the keyboard, the screen and the screen for errors.

Looking ahead

The next lecture continues with files, and looks at binary files: a different way of storing data, which is not meant to be read in a text editor.

Exercises

1. Write a program that reads `movies.txt` with `fgets` and counts how many lines it has, using `while (fgets(...) != NULL)`. Then change it to use `feof` instead, and check that you get the same count. What count do you get if you use the first, buggy loop from this lecture, with the `fgets` followed immediately by the `printf` inside the loop? Explain the difference.
2. Make a file called `grades.txt` with a student name and a mark on each line, like `Thandi 78`. Write a program that reads every line with `fscanf` and prints the average mark.
3. Run the file-writing program from the start of this lecture twice, first with `"w"` and then with `"a"`. How does the file differ after the second run? What would you expect if you opened the same file with `"r"` and tried to write to it?
4. Write a program that copies one text file to another, line by line. The names of both files should be read from the user. Be careful which file you open with which mode, and which one you might destroy.